

## Load Datasets

```
In [18]: import os
from datasets import load_dataset, load_from_disk

# Set your base data directory
CWD = os.getcwd()
DATA_DIR = os.path.join(CWD, "data")

# For CommonsenseQA
CSQA_PATH = os.path.join(DATA_DIR, "commonsenseqa_local")
if os.path.exists(CSQA_PATH):
    csqa = load_from_disk(CSQA_PATH)
else:
    csqa = load_dataset("commonsense_qa", cache_dir=DATA_DIR)
    csqa.save_to_disk(CSQA_PATH)

# For SNLI
SNLI_PATH = os.path.join(DATA_DIR, "snli_local")
if os.path.exists(SNLI_PATH):
    snli = load_from_disk(SNLI_PATH)
else:
    snli = load_dataset("snli", cache_dir=DATA_DIR)
    snli.save_to_disk(SNLI_PATH)
```

## Preview Dataset

```
In [19]: # Use the basic SNLI dataset to start
print(snli['train'][0])

# You can generate explanations for these examples
# This actually aligns with your research - creating better explanations!
```

```
{'premise': 'A person on a horse jumps over a broken down airplane.', 'hypothesis':
'A person is training his horse for a competition.', 'label': 1}
```

```
In [20]: # Look at the dataset structure
print("CommonsenseQA structure:")
print(csqa)
print("\nSample CommonsenseQA example:")
example = csqa['train'][0]
for key, value in example.items():
    print(f"{key}: {value}")
```

CommonsenseQA structure:

```
DatasetDict({
  train: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 9741
  })
  validation: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1221
  })
  test: Dataset({
    features: ['id', 'question', 'question_concept', 'choices', 'answerKey'],
    num_rows: 1140
  })
})
```

Sample CommonsenseQA example:

id: 075e483d21c29a511267ef62bedc0461

question: The sanctions against the school were a punishing blow, and they seemed to what the efforts the school had made to change?

question\_concept: punishing

choices: {'label': ['A', 'B', 'C', 'D', 'E'], 'text': ['ignore', 'enforce', 'authoritarian', 'yell at', 'avoid']}

answerKey: A

```
In [21]: # Look at SNLI structure
print("\nSNLI structure:")
print(snli)
print("\nSample SNLI example:")
example = snli['train'][0]
for key, value in example.items():
    print(f"{key}: {value}")
```

SNLI structure:

```
DatasetDict({
  test: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 10000
  })
  validation: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 10000
  })
  train: Dataset({
    features: ['premise', 'hypothesis', 'label'],
    num_rows: 550152
  })
})
```

Sample SNLI example:

premise: A person on a horse jumps over a broken down airplane.

hypothesis: A person is training his horse for a competition.

label: 1

## Sample the datasets

```

In [22]: import random
random.seed(42)

NUM_CSQA = 300
NUM_SNLI = 300

# --- CommonsenseQA sampling ---
csqa_indices = random.sample(range(len(csqa['train'])), NUM_CSQA)
commonsense_examples = []
for i in csqa_indices:
    example = csqa['train'][i]
    commonsense_examples.append({
        'id': i,
        'question': example['question'],
        'choices': example['choices']['text'],
        'answer': example['answerKey'],
        'dataset': 'csqa'
    })

# --- SNLI sampling (valid only) ---
valid_snli_indices = [i for i, ex in enumerate(snli['train']) if ex['label'] != -1]
snli_indices = random.sample(valid_snli_indices, NUM_SNLI)
snli_examples = []
for i in snli_indices:
    example = snli['train'][i]
    snli_examples.append({
        'id': i,
        'premise': example['premise'],
        'hypothesis': example['hypothesis'],
        'label': example['label'],
        'dataset': 'snli'
    })

```

```

In [25]: import os
import json

# Directories and constants
SAMPLE_DIR = os.path.join(DATA_DIR, "sampled_examples")
os.makedirs(SAMPLE_DIR, exist_ok=True)

LABEL_NAMES = ['entailment', 'neutral', 'contradiction']

# Save separate files as well (optional)
csqa_sample_path = os.path.join(SAMPLE_DIR, "csqa_sample.json")
with open(csqa_sample_path, "w", encoding="utf-8") as f:
    json.dump(commonsense_examples, f, indent=4, ensure_ascii=False)
print(f"✅ Saved {len(commonsense_examples)} CommonsenseQA examples to: {csqa_sample_path}")

snli_sample_path = os.path.join(SAMPLE_DIR, "snli_sample.json")
with open(snli_sample_path, "w", encoding="utf-8") as f:
    json.dump(snli_examples, f, indent=4, ensure_ascii=False)
print(f"✅ Saved {len(snli_examples)} SNLI examples to: {snli_sample_path}")

# --- Save pretty TXT for annotation/human reading ---
csqa_sample_txt_path = os.path.join(SAMPLE_DIR, "csqa_sample.txt")

```

```

with open(csqa_sample_txt_path, "w", encoding="utf-8") as f:
    f.write("=== CommonsenseQA Sample ===\n\n")
    for ex in commonsense_examples:
        f.write(f"ID: {ex['id']}\n")
        f.write(f"Question: {ex['question']}\n")
        f.write(f"Choices: {ex['choices']}\n")
        f.write(f"Answer: {ex['answer']}\n")
        f.write("-" * 60 + "\n")
print(f"✅ Saved {len(commonsense_examples)} readable subset to: {csqa_sample_txt_path}")

snli_sample_txt_path = os.path.join(SAMPLE_DIR, "snli_sample.txt")
with open(snli_sample_txt_path, "a", encoding="utf-8") as f:
    f.write("\n=== SNLI Sample ===\n\n")
    for ex in snli_examples:
        f.write(f"ID: {ex['id']}\n")
        f.write(f"Premise: {ex['premise']}\n")
        f.write(f"Hypothesis: {ex['hypothesis']}\n")
        f.write(f"Label: {LABEL_NAMES[ex['label']]\n")
        f.write("-" * 60 + "\n")
print(f"✅ Saved {len(snli_examples)} readable subset to: {snli_sample_txt_path}")

```

- ✅ Saved 300 CommonsenseQA examples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\sampled\_examples\csqa\_sample.json
- ✅ Saved 300 SNLI examples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\sampled\_examples\snli\_sample.json
- ✅ Saved 300 readable subset to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\sampled\_examples\csqa\_sample.txt
- ✅ Saved 300 readable subset to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\sampled\_examples\snli\_sample.txt

## Preprocess the sample

### ✅ Step 1: Load Your Local Sample Files

```

In [26]: import json

# Load your pre-saved 300 examples
with open(snli_sample_path, "r", encoding="utf-8") as f:
    snli_data = json.load(f)

with open(csqa_sample_path, "r", encoding="utf-8") as f:
    csqa_data = json.load(f)

```

### 🔗 Step 2: Preprocess for Attribution

```

In [28]: # Data directory for saving processed examples
PROCESSED_DIR = os.path.join(DATA_DIR, "processed_examples")
os.makedirs(PROCESSED_DIR, exist_ok=True)

```

```

In [ ]: # 📄 SNLI: Convert Premise + Hypothesis
processed_snli_path = os.path.join(PROCESSED_DIR, "processed_snli.json")
processed_snli = []
for ex in snli_data:
    text = f"{ex['premise']} [SEP] {ex['hypothesis']}"

```

```

processed_snli.append({
    "id": ex["id"],
    "input_text": text,
    "label": ex["label"],
    "dataset": "snli"
})

# Save to JSON for LIME/SHAP usage
with open(processed_snli_path, "w", encoding="utf-8") as f:
    json.dump(processed_snli, f, indent=2, ensure_ascii=False)
print(f"✅ Saved processed SNLI examples to: {processed_snli_path}")

```

✅ Saved processed SNLI examples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\processed\_examples\processed\_snli.json

```

In [30]: # 📄 CommonsenseQA: Store as Question + Choices
import string

processed_csqa_path = os.path.join(PROCESSED_DIR, "processed_csqa.json")
processed_csqa = []
choice_map = list(string.ascii_uppercase)

for ex in csqa_data:
    processed_csqa.append({
        "id": ex["id"],
        "question": ex["question"],
        "choices": ex["choices"],
        "correct_choice_idx": choice_map.index(ex["answer"]),
        "dataset": "csqa"
    })

with open(processed_csqa_path, "w", encoding="utf-8") as f:
    json.dump(processed_csqa, f, indent=2, ensure_ascii=False)
print(f"✅ Saved processed CommonsenseQA examples to: {processed_csqa_path}")

```

✅ Saved processed CommonsenseQA examples to: c:\Users\Work\OneDrive\Desktop\ChatGPT\Research\project\data\processed\_examples\processed\_csqa.json

## f

```

In [31]: import os
import json
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from lime.lime_text import LimeTextExplainer

```

```

In [32]: EXPLANATIONS_DIR = os.path.join(PROCESSED_DIR, "snli_lime_outputs")
os.makedirs(EXPLANATIONS_DIR, exist_ok=True)

```

```

In [ ]:

```